

views::chunk_by

Document #: P2443R1
Date: 2021-11-19
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

- [1 Revision history](#)
- [2 Abstract](#)
- [3 Example](#)
- [4 Design](#)
 - [4.1 Predicate, not equivalence](#)
 - [4.2 No input ranges](#)
 - [4.3 Properties](#)
 - [4.4 Caching](#)
 - [4.5 Implementation experience](#)
- [5 Wording](#)
 - [5.1 Addition to `<ranges>`](#)
 - [5.2 `chunk\_by`](#)
 - [24.7.? `Chunk by view` \[`range.chunk.by`\]](#)
 - [5.3 Feature-test macro](#)
- [6 References](#)

§ 1 Revision history

— R1: Incorporated LWG review feedback.

§ 2 Abstract

This paper proposes the range adaptor `views::chunk_by` as described in section 4.3 of [\[P2214R1\]](#).

§ 3 Example

```
std::vector v = {1, 2, 2, 3, 0, 4, 5, 2};  
fmt::print("{}\n", v | std::views::chunk_by(ranges::less_equal{})); // [[1, 2, 2,  
3], [0, 4, 5], [2]]
```

§ 4 Design

Section 4.3 of [P2214R1] contains an extensive discussion of `chunk_by` and similar range algorithms in numerous other languages. The discussion here assumes familiarity with that paper.

§ 4.1 Predicate, not equivalence

D's `chunkBy` [requires](#) the predicate to be an equivalence relation. The initial implementation of `range-v3`'s `group_by` appeared to tacitly assume this as well, as it was not functional `if pred(*first, *first). returns false`.

There does not appear to be a compelling reason to require an equivalence relation. Indeed, D provides virtually the same functionality for general predicates under the name `splitWhen`. The `chunk_by` proposed in this paper therefore does not require equivalence.

§ 4.2 No input ranges

Since `chunk_by` needs to evaluate the predicate on adjacent elements, it requires forward ranges. Caching elements is not considered a viable approach, essentially for the reasons discussed in section 4.3.4 of [P2321R2].

§ 4.3 Properties

`chunk_by` is bidirectional if the underlying range is; otherwise, it is forward. It is common if the underlying range is. It's never borrowed or sized.

§ 4.4 Caching

Similar to `split`, `chunk_by` calculates the end of the first range in its `begin` and caches it to meet the amortized $O(1)$ requirement. That means that it does not support `const-iteration`.

§ 4.5 Implementation experience

`views::group_by` has long been implemented in `range-v3`, with the difference that it compares against the first element of the group instead of the previous element. A `views::chunk_by` with the semantics proposed in this paper (except for bidirectional iteration support) has recently been added to `range-v3`. I also have implemented a version that directly follows the proposed wording below without issue.

§ 5 Wording

§ 5.1 Addition to `<ranges>`

Add the following to 24.2 [ranges.syn], header `<ranges>` synopsis:

```
// [...]  
namespace std::ranges {  
    // [...]
```

```

// [range.chunk.by], chunk by view
template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>>
    Pred>
    requires view<V> && is_object_v<Pred>
class chunk_by_view;

namespace views {
    inline constexpr unspecified chunk_by = unspecified;
}
}

```

§ 5.2 chunk_by

Add the following subclause to 24.7 [\[range.adaptors\]](#).

§ 24.7.? Chunk by view [\[range.chunk.by\]](#)

§ 24.7.?.1 Overview [\[range.chunk.by.overview\]](#)

- 1 chunk_by_view takes a view and a predicate, and splits the view into subranges between each pair of adjacent elements for which the predicate returns `false`.
- 2 The name `views::chunk_by` denotes a range adaptor object (24.7.2 [\[range.adaptor.object\]](#)). Given subexpressions `E` and `F`, the expression `views::chunk_by(E, F)` is expression-equivalent to `chunk_by_view(E, F)`.

[Example 1:

```

vector v = {1, 2, 2, 3, 0, 4, 5, 2};

for (auto r : v | views::chunk_by(ranges::less_equal{})) {
    cout << '[';
    auto sep = "";
    for(auto i : r) {
        cout << sep << i;
        sep = ", ";
    }
    cout << "]" << " ";
}
// prints: [1, 2, 2, 3] [0, 4, 5] [2]

```

— end example]

§ 24.7.?.2 Class template chunk_by_view [\[range.chunk.by.view\]](#)

```

namespace std::ranges {
    template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>>
        Pred>
        requires view<V> && is_object_v<Pred>
    class chunk_by_view : public view_interface<chunk_by_view<V, Pred>>{
        V base_ = V(); // exposition only
    };
}

```

```

copyable-box<Pred> pred_ = Pred();           // exposition only

class iterator;                               // exposition only

public:
    chunk_by_view() requires default_initializable<V> && default_initializable<Pred> =
        default;
    constexpr explicit chunk_by_view(V base, Pred pred);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr const Pred& pred() const;

    constexpr iterator begin();
    constexpr auto end();

    constexpr iterator_t<V> find-next(iterator_t<V>); // exposition only
    constexpr iterator_t<V> find-prev(iterator_t<V>) requires bidirectional_range<V>; //
        exposition only
};

template<class R, class Pred>
    chunk_by_view(R&&, Pred) -> chunk_by_view<views::all_t<R>, Pred>;
}

```

```
constexpr explicit chunk_by_view(V base, Pred pred);
```

- 1 *Effects:* Initializes `base_` with `std::move(base)` and `pred_` with `std::move(pred)`.

```
constexpr const Pred& pred() const;
```

- 2 *Effects:* Equivalent to `return *pred_;`

```
constexpr iterator begin();
```

- 3 *Preconditions:* `pred_.has_value()` is `true`.
- 4 *Returns:* `iterator(*this, ranges::begin(base_), find-next(ranges::begin(base_)))`.
- 5 *Remarks:* In order to provide the amortized constant-time complexity required by the range concept, this function caches the result within the `chunk_by_view` for use on subsequent calls.

```
constexpr auto end();
```

- 6 *Effects:* Equivalent to:

```

if constexpr (common_range<V>) {
    return iterator(*this, ranges::end(base_), ranges::end(base_));
}
else {

```

```
    return default_sentinel;
}
```

```
constexpr iterator_t<V> find-next(iterator_t<V> current); // exposition only
```

7 *Preconditions:* `pred_.has_value()` is `true`.

8 *Returns:* `ranges::next(ranges::adjacent_find(current, ranges::end(base_), not_fn(ref(*pred_))), 1, ranges::end(base_))`.

```
constexpr iterator_t<V> find-prev(iterator_t<V> current) requires
    bidirectional_range<V>; // exposition only
```

9 *Preconditions:*

(9.1) — `current` is not equal to `ranges::begin(base_)`.

(9.2) — `pred_.has_value()` is `true`.

10 *Returns:* An iterator `i` in the range `[ranges::begin(base_), current)` such that:

(10.1) — `ranges::adjacent_find(i, current, not_fn(ref(*pred_)))` is equal to `current`; and

(10.2) — if `i` is not equal to `ranges::begin(base_)`, then `bool(involve(*pred_, *ranges::prev(i), *i))` is `false`.

[Drafting note: Reference implementation:

```
using namespace std::placeholders;
reverse_view rv(subrange(ranges::begin(base_), current));
return ranges::prev(ranges::adjacent_find(rv, not_fn(bind(ref(*pred_), _2,
    _1))).base(),
    1, ranges::begin(base_));
```

]

§ 24.7.3 Class `chunk_by_view::iterator` [range.chunk.by.iter]

```
namespace std::ranges {
    template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>>
        Pred>
        requires view<V> && is_object_v<Pred>
        class chunk_by_view<V, Pred>::iterator {
            chunk_by_view* parent_ = nullptr; // exposition only
            iterator_t<V> current_ = iterator_t<V>(); // exposition only
            iterator_t<V> next_ = iterator_t<V>(); // exposition only

            constexpr iterator(chunk_by_view& parent, iterator_t<V> current, iterator_t<V>
                next); // exposition only

        public:
            using value_type = subrange<iterator_t<V>>;
            using difference_type = range_difference_t<V>;
            using iterator_category = input_iterator_tag;
```

```

using iterator_concept = see below;

iterator() = default;

constexpr value_type operator*() const;
constexpr iterator& operator++();
constexpr iterator operator++(int);

constexpr iterator& operator--() requires bidirectional_range<V>;
constexpr iterator operator--(int) requires bidirectional_range<V>;

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
};
}

```

1 `iterator::iterator_concept` is defined as follows:

- .1) — If `V` models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- .2) — Otherwise, `iterator_concept` denotes `forward_iterator_tag`.

```
constexpr iterator(chunk_by_view& parent, iterator_t<V> current, iterator_t<V> next);
```

- 2 *Effects*: Initializes `parent_` with `addressof(parent)`, `current_` with `current`, and `next_` with `next`.

```
constexpr value_type operator*() const;
```

- 3 *Preconditions*: `current_` is not equal to `next_`.
- 4 *Returns*: `subrange(current_, next_)`.

```
constexpr iterator& operator++();
```

- 5 *Preconditions*: `current_` is not equal to `next_`.
- 6 *Effects*: Equivalent to:

```

current_ = next_;
next_ = parent_ -> find-next(current_);
return *this;

```

```
constexpr iterator operator++(int);
```

- 7 *Effects*: Equivalent to:

```

auto tmp = *this;
++*this;
return tmp;

```

```
constexpr iterator& operator--() requires bidirectional_range<V>;
```

- 8 *Effects*: Equivalent to:

```
next_ = current_;
current_ = parent_->find-prev(next_);
return *this;
```

```
constexpr iterator operator--(int) requires bidirectional_range<V>;
```

9 *Effects:* Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

10 *Returns:* `x.current_ == y.current_`.

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

11 *Returns:* `x.current_ == x.next_`.

§ 5.3 Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_chunk_by 20XXXXL // also in <ranges>
```

§ 6 References

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021. A Plan for C++23 Ranges.

<https://wg21.link/p2214r1>

[P2321R2] Tim Song. 2021-06-11. zip.

<https://wg21.link/p2321r2>